

Curso de Jogos Digitais

Disciplina de Computação Gráfica /

Camera / Texturas /Objetos

Aula 12

Professor: André Flores dos Santos



Movimentos da Camera

Nesta aula, trabalharemos com câmera controlável em um ambiente 3D usando OpenGL + Pygame. O foco está nos movimentos da câmera com teclado e mouse para explorar um cenário com cubo(s) texturizado(s).

Movimentos da Camera

- Objetivo principal:
 - Controlar movimentos para giros horizontal (Yaw: esquerda ou direita) e vertical (Pitch: para cima ou para baixo)

Por que precisamos de yaw e pitch?

Imagine que você está de frente para um cubo em um espaço 3D. Agora:

- Se você gira o mouse para a direita → seu "rosto" gira (isso é o yaw).
- Se você inclina o mouse para cima → seu rosto aponta para cima (isso é o pitch).

Esses dois movimentos definem a direção em que a câmera está olhando.

Movimentos da Camera

No código, controlamos a câmera usando dois ângulos:

- **Yaw**: rotação horizontal da câmera.
- **Pitch**: rotação vertical (olhar para cima/baixo).

- Utilizamos trigonometria para transformar os ângulos da câmera em um vetor de direção 3D:

$$\text{dir_x} = \cos(\text{pitch}) * \sin(\text{yaw})$$

$$\text{dir_y} = \sin(\text{pitch})$$

$$\text{dir_z} = \cos(\text{pitch}) * \cos(\text{yaw})$$

Esses valores determinam para onde a câmera está olhando no espaço 3D.

Movimentos da Camera

Exemplo com `yaw = 0` e `pitch = 0`

Neste exemplo, a câmera está reta olhando para frente.

Vamos calcular o vetor direção com `yaw = 0` e `pitch = 0`:

`yaw = 0`

`pitch = 0`

`rad_yaw = math.radians(0) # 0 rad`

`rad_pitch = math.radians(0) # 0 rad`

`dir_x = cos(0) * sin(0) = 1 * 0 = 0`

`dir_y = sin(0) = 0`

`dir_z = cos(0) * cos(0) = 1 * 1 = 1`

A câmera está olhando para frente no eixo Z positivo $\rightarrow (0, 0, 1)$

Movimentos da Camera

Exemplo com `yaw = 90°` e `pitch = 0`

`yaw = 90`

`pitch = 0`

`rad_yaw = math.radians(90) = 1.57 rad`

`rad_pitch = math.radians(0) = 0`

`dir_x = cos(0) * sin(90°) = 1 * 1 = 1`

`dir_y = sin(0) = 0`

`dir_z = cos(0) * cos(90°) = 1 * 0 = 0`

A câmera está olhando para o eixo X positivo (para a esquerda da tela).

Movimentos da Camera

Efeito do yaw no sentido da câmara

Yaw	Direção	Significado (posição da câmara)
0°	(0, 0, 1)	Olhando para frente no eixo Z
90°	(1, 0, 0)	Olhando para a esquerda (X positivo)
-90°	(-1, 0, 0)	Olhando para a direita (X negativo)
180°	(0, 0, -1)	Olhando para trás (Z negativo)

Movimentos da Camera

□ Como transformar yaw/pitch em vetor 3D?

A gente usa fórmulas baseadas em trigonometria, que vêm da conversão de ângulos esféricos para coordenadas cartesianas. Isso é algo comum em 3D e jogos.

Fórmulas usadas:

$$\text{dir_x} = \cos(\text{pitch}) * \sin(\text{yaw})$$
$$\text{dir_y} = \sin(\text{pitch})$$
$$\text{dir_z} = \cos(\text{pitch}) * \cos(\text{yaw})$$

Essas fórmulas transformam os ângulos em um vetor tridimensional (x, y, z) que aponta na direção que a câmera está olhando.

Esses vetores de direção são usados na função que utilizamos para controlar a câmera.

Exemplo `gluLookAt(eyeX, eyeY, eyeZ, X, Y, Z, 0, 1, 0)`

Movimentos da Camera

A ideia do movimento lateral (strafe)

Quando você anda para os lados com a câmera (como em um jogo em 1ª pessoa), você está andando em uma direção perpendicular à direção que está olhando.

Isso significa:

- Se você está olhando para frente, andar para o lado é como andar 90 para a esquerda ou direita.

Movimentos da Camera

`yaw = 0`

`pitch = 0`

`rad_yaw = math.radians(0) # 0 rad`

`rad_pitch = math.radians(0) # 0 rad`

`dir_x = cos(0) * sin(0) = 1 * 0 = 0`

`dir_y = sin(0) = 0`

`dir_z = cos(0) * cos(0) = 1 * 1 = 1`

Quando `yaw = 0`, a câmera olha para frente no eixo Z.

Se pressionarmos **tecla A** (mover para a esquerda):

`camera_x += dir_z * 0.05 → camera_x += 1 * 0.05 = +0.05`

`camera_z -= dir_x * 0.05 → camera_z -= 0 * 0.05 = 0`

A câmera se move para a esquerda da tela (**X positivo**).

Se pressionarmos **tecla D** (mover para a direita):

`camera_x -= dir_z * 0.05 → camera_x -= 1 * 0.05 = -0.05`

`camera_z -= dir_x * 0.05 → camera_z -= 0 * 0.05 = 0`

A câmera vai para a direita da tela (**X negativo**).

Movimentos da Camera

yaw = 90

pitch = 0

rad_yaw = math.radians(90) = 1.57 rad

rad_pitch = math.radians(0) = 0

dir_x = cos(0) * sin(90°) = 1 * 1 = 1

dir_y = sin(0) = 0

dir_z = cos(0) * cos(90°) = 1 * 0 = 0

Quando **yaw = 90**, a câmera olha esquerda ou na direção x+.

Se pressionarmos **tecla A:**

camera_x += dir_z * 0.05 → camera_x += 0 = 0

camera_z -= dir_x * 0.05 → camera_z -= 1 * 0.05 = -0.05

A câmera se move para trás no Z- (direita da tela)

Se pressionarmos **tecla D:**

camera_x += dir_z * 0.05 → camera_x += 0 = 0

camera_z += dir_x * 0.05 → camera_z += 1 * 0.05 = +0.05

A câmera se move para frente no Z+ (esquerda da tela)

Movimentos da Camera

Por que usamos X e Z ao mesmo tempo?

Como a câmera pode estar rotacionada (yaw), mover-se apenas em X ou apenas em Z não seria suficiente.

Precisamos combinar `dir\_x` e `dir\_z` para andar exatamente 90° da direção atual.

Exemplo:

- Se a câmera estiver olhando para frente no eixo Z (yaw = 0°):
 - A move para esquerda (X positivo)
 - D move para direita (X negativo)
- Se a câmera estiver olhando para a direita (yaw = 90°):
 - A move para trás (Z negativo)
 - D move para frente (Z positivo)

Portanto, sempre usamos os dois eixos combinados.

Movimentos da Camera

Conclusão

- O vetor de direção (dir_x , dir_z) depende dos ângulos yaw/pitch (giro da câmera).
 - Para se mover lateralmente com realismo, é preciso andar em um vetor perpendicular à direção da câmera.
 - No código, isso é feito com as combinações de ``dir_z`` e ``dir_x``.
 - Esse cálculo garante que o movimento 'A' e 'D' funcione corretamente, não importa para onde a câmera está olhando.
- **Código para testar em anexo na aula de hoje:**
- [exemplo_opengl_textura_camera_iluminacao_Parte_I.rar](#) (compactado)

PARTE II

PARTE II – IMPORTAÇÃO DE OBJETOS 3D NO OPENGL

De onde baixar arquivos ‘.obj’ para usar no OpenGL com PyOpenGL

Aqui vai uma lista dos principais sites onde você pode baixar modelos 3D (em formatos como OBJ, FBX, STL, etc.) prontos para importar em OpenGL:

- **Free3D** (free3d.com)

Modelos gratuitos e pagos em OBJ, FBX, 3DS, BLEND. Excelente para testes rápidos.

- **Sketchfab** (sketchfab.com)

Milhares de modelos com visualização em tempo real. Muitos são gratuitos para download (procure a aba “Download” e filtro “Free”).

- **TurboSquid** (turbosquid.com)

Um dos maiores acervos — modelos gratuitos e profissionais em vários formatos.

- **CGTrader** (cgtrader.com)

Modelos pagos e free, ótima qualidade. Possui filtros por licença e formato.

- **Blend Swap** (blendswap.com)

Focado em arquivos Blender (.blend), mas a maioria pode ser exportada para OBJ.

- **Thingiverse** (thingiverse.com)

Biblioteca enorme de modelos para impressão 3D (STL), que você pode converter/exportar para OBJ.

- **3D Warehouse** (3dwarehouse.sketchup.com)

Modelos SketchUp prontos; exporte como COLLADA (.dae) ou OBJ.

- **Clara.io** (clara.io)

Editor/modelador online com repositório de modelos OBJ/FBX gratuitos.

- **Poly Haven** (polyhaven.com)

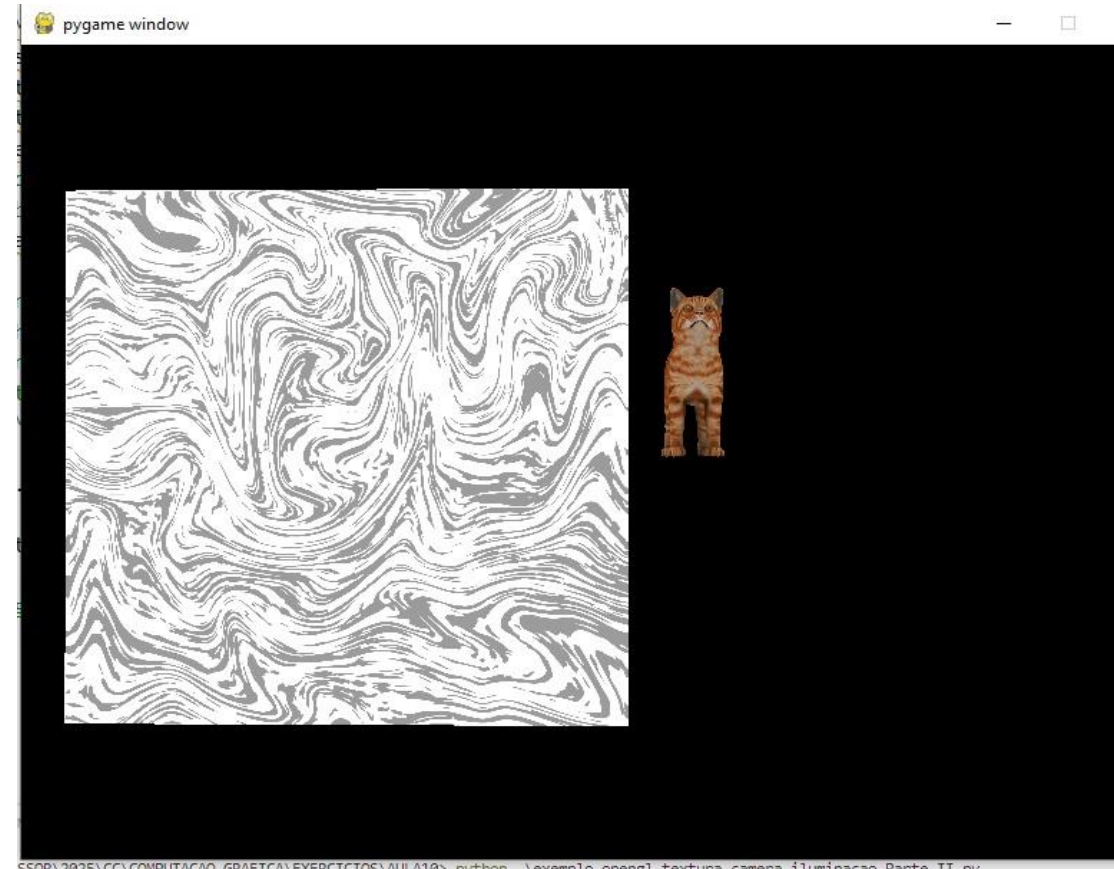
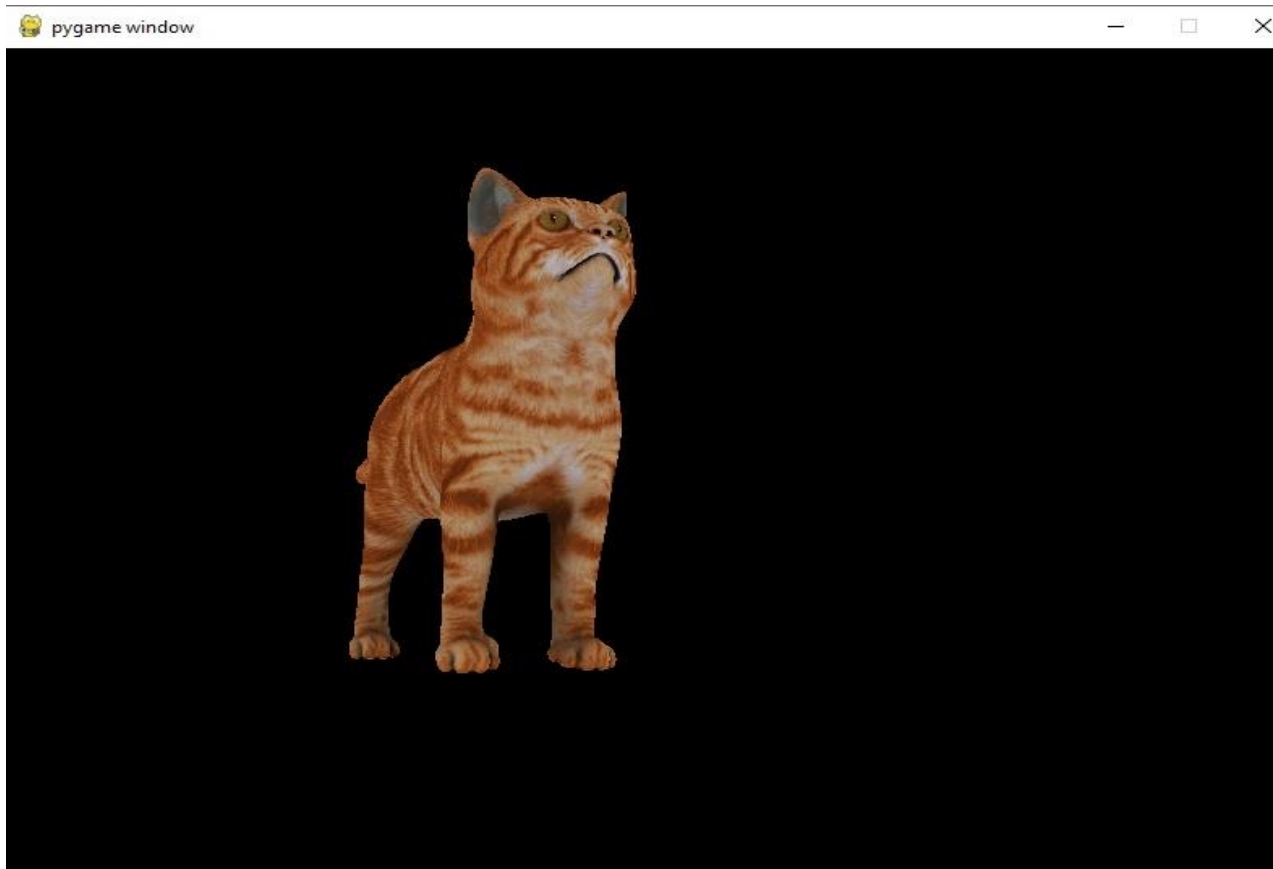
Além de HDRIs e texturas, oferece também alguns modelos 3D gratuitos em OBJ/GLTF.

- **OpenGameArt** (opengameart.org)

Modelos otimizados para jogos, incluindo pacotes low-poly em OBJ.

De onde baixar arquivos '.obj' para usar no OpenGL com PyOpenGL

- Exemplo de importação do Gato 3D.
- Link para download: <https://free3d.com/pt/3d-model/cat-v1--522281.html>



De onde baixar arquivos `‘.obj’` para usar no OpenGL com PyOpenGL

- Código base (em anexo) para iniciar o projeto está disponível para download na aula de hoje.

Nome: exemplo_opengl_textura_camera_iluminacao_Parte_II.py

Adaptem o nome da textura na configuração de importação e o caminho da pasta de onde é carregado o objeto.

Exercícios

- 1) Descrição da atividade em anexo na aula de hoje.

Referências e material de apoio

Material do Professor Guilherme Chagas Kurtz, 2023.

GOMES, Jonas; VELHO, Luiz. Computação gráfica. Rio de Janeiro: Impa, 1998.

HEARN, Donald; Baker, M. Pauline. Computer graphics: C version. London: Prentice Hall, 1997.

HETEM JUNIOR, Annibal. Computação gráfica. Rio de Janeiro, RJ: LTC, 2006. 161 p. (Coleção Fundamentos de Informática).

HILL Jr, Francis S. Computer graphics using open GL. New Jersey: Prentice Hall, 2001.

WATT, Alan. 3D computer graphics. Harlow: Addison-Wesley, 2000

Thank you for your attention!!



Email: andre.flores@ufn.edu.br